

Desafio Técnico — Analista de Dados Pleno

Lakehouse On-Premises (MinIO · Iceberg · Spark · Trino · Airflow)

1. Contexto

Nossa plataforma de dados roda **on-premises**, não em nuvem gerenciada:

Camada	Tecnologia
Object storage	MinIO (API compatível com S3)
Table format	Apache Iceberg
Processamento	Apache Spark / PySpark
Engine OLAP	Trino
Orquestração	Apache Airflow
Fontes	APIs REST, PostgreSQL, arquivos

O objetivo deste desafio é ver **como você pensa e constrói um pipeline de ponta a ponta** nesse stack — não decorar sintaxe de Iceberg. Se você nunca usou Iceberg ou Trino, tudo bem: o ambiente vem pronto e a documentação oficial é permitida (e esperada).

Tempo estimado: 6 a 8 horas de trabalho efetivo. **Prazo de entrega:** 3 dias corridos a partir do recebimento.

Se você chegar no limite de tempo sem terminar tudo, **entregue o que estiver funcionando** e descreva no README o que faltou e como faria. Entrega incompleta bem documentada vale mais do que entrega completa sem explicação.

2. O que você vai construir

None



3. Ambiente

Você recebe um repositório com o ambiente inteiro pronto (MinIO, catálogo Iceberg REST, Spark, Trino, PostgreSQL e a API mock). Três comandos:

```
Shell
cp .env.example .env
make up          # gera os dados, sobe tudo e valida o ambiente
make check       # revalida quando quiser
make help        # todos os atalhos
```

O `make check` faz um teste real de ponta a ponta: escreve uma tabela Iceberg pelo Spark, no MinIO, e lê pelo Trino. Se ele passar, o ambiente está íntegro e qualquer problema dali em diante é do seu código.

Serviço	Do seu terminal	De dentro dos containers	Credenciais
MinIO Console	http://localhost:9001	—	admin / minioadmin
MinIO (S3 API)	http://localhost:9000	http://minio:9000	idem
PostgreSQL	localhost:5432	postgres:5432	app / app, db crm
Mock API	http://localhost:8000/docs	http://mock-api:8000	header X-API-Key: desafio-2026
Trino	http://localhost:8080	trino:8080	qualquer usuário
Airflow (opcional)	http://localhost:8081	—	make airflow

Atalhos: `make spark` (shell no container, com o repo em `/home/iceberg/work`), `make pyspark`, `make trino`, `make psql`, `make logs`, `make clean`.

O catálogo Iceberg se chama `lakehouse` e já está configurado nos dois motores — tabela criada no Spark aparece no Trino na hora. O warehouse fica em `s3://lakehouse/warehouse` (gerenciado pelo Iceberg) e a raw zone é sua, em `s3://lakehouse/raw/`.

Se precisar ajustar portas, versões de imagem ou memória do Spark para rodar na sua máquina, **pode alterar o compose** — só registre o que mudou.

4. Fontes de dados

4.1 Mock API — `GET /events`

Eventos de uso do produto e de atendimento ao cliente.

None

`GET /events?since=2026-01-01T00:00:00Z&page=1&page_size=500`
Header: X-API-Key: desafio-2026

Resposta:

JSON

```
{
  "page": 1,
  "page_size": 500,
  "total_pages": 37,
  "data": [
    {
      "event_id": "e_9f3a...",
      "customer_id": "c_0142",
      "event_type": "ticket_opened",
      "occurred_at": "2026-03-11T14:02:11Z",
      "updated_at": "2026-03-11T14:02:11Z",
      "channel": "email",
      "properties": { "ticket_id": "t_881", "priority": "high", "agent_id": "a_12"
    }
  ]
}
```

Características **reais e intencionais** dessa API (todas presentes nos dados):

- Paginação obrigatória; retorna 429 se você passar de ~10 requisições/segundo.
- Falha intermitente com 503 em cerca de 2% das chamadas.
- Registros **duplicados** entre páginas e entre execuções.
- Registros **corrigidos**: o mesmo `event_id` pode reaparecer depois com `updated_at` mais recente e valores diferentes.
- **Late arrival**: eventos com `occurred_at` de até 3 dias atrás chegam em lotes posteriores.
- `properties` é um JSON semiestruturado — as chaves variam por `event_type`.
- A partir de determinada data, um campo novo aparece no payload (**schema drift**). Seu pipeline não pode quebrar por causa disso.
- Alguns registros têm `customer_id` nulo ou inexistente na dimensão.

4.2 PostgreSQL — `crm.customers`

SQL

<code>customer_id</code>	<code>text primary key</code>
<code>company_name</code>	<code>text</code>
<code>plan</code>	<code>text</code> <code>-- 'free' 'pro' 'enterprise'</code>
<code>segment</code>	<code>text</code>
<code>signup_date</code>	<code>date</code>
<code>country</code>	<code>text</code>
<code>is_active</code>	<code>boolean</code>
<code>updated_at</code>	<code>timestampz</code> <code>-- muda quando o cadastro é alterado</code>

Entre a carga inicial e a segunda carga, alguns clientes **mudam de plano**, outros são desativados e outros são criados. O histórico dessas mudanças importa para o negócio.

4.3 Os dois batches

As fontes começam no **batch 1**. Quando seu pipeline estiver rodando, avance para o batch 2 — é assim que simulamos o tempo passando entre duas execuções:

Shell

```
make batch-state    # em qual batch as fontes estão
make batch2         # libera o batch 2 na API e aplica as mudanças no Postgres
```

O batch 2 traz eventos novos, correções de eventos antigos, registros atrasados, o campo novo do schema drift e as mudanças de cadastro.

A sequência exata que vamos executar para avaliar sua entrega:

None

```
pipeline (batch 1) → pipeline de novo → make batch2 → pipeline → pipeline de novo
```

Em nenhum desses cinco passos pode haver duplicação, perda de dado ou quebra.

5. Entregas obrigatórias

Parte 1 — Ingestão (raw zone)

Script Python que extrai da API e do PostgreSQL e grava em `s3a://lakehouse/raw/`.

Requisitos:

- Preservar o payload **como veio** (sem transformação de negócio na raw).
- Particionar por fonte e data de ingestão, ex.:
`raw/events/ingestion_date=2026-03-11/part-000.json.gz`.
- Tratar paginação, 429 e 503 (retry com backoff).
- Ser **idempotente**: rodar duas vezes o mesmo dia não pode corromper a raw nem duplicar arquivos.
- Ingestão **incremental** por `updated_at`, com controle de watermark (onde você guarda esse estado é decisão sua — justifique).

- Logs estruturados com contagem de registros, páginas lidas, retries e duração.

Parte 2 — Bronze e Silver com PySpark + Iceberg

Bronze (`lakehouse.bronze.events`, `lakehouse.bronze.customers`)

- Fiel à raw, tipado, com colunas de controle (`_ingested_at`, `_source_file`, `_batch_id`).
- Append-only.

Silver (`lakehouse.silver.events`, `lakehouse.silver.customers`)

- Deduplicação por chave de negócio mantendo a **última versão** de cada registro (`MERGE INTO`).
- Tipos corretos, timestamps normalizados em UTC, `properties` acessível para consulta.
- Estratégia de particionamento justificada (e você deve explicar por que escolheu `partitioned by` daquele jeito em vez de outro).
- `silver.customers` deve permitir responder "**qual era o plano do cliente na data do evento**" — a forma de modelar isso é sua decisão de projeto.

Critério duro: rodar o pipeline completo **duas vezes seguidas** tem que produzir exatamente o mesmo resultado. Vamos testar isso.

Parte 3 — Gold + Trino

Crie as tabelas gold que julgar necessárias e responda via **SQL no Trino**:

1. **Top 10 clientes** por volume de eventos nos últimos 30 dias da base, com nome da empresa, plano e segmento.
2. **Tempo médio até a primeira resposta** (`ticket_opened` → primeiro `ticket_replied` do mesmo `ticket_id`), por plano e por mês. Trate tickets sem resposta explicitamente.
3. **Retenção por coorte**: percentual de clientes com pelo menos 1 evento no mês, agrupados pelo mês de `signup_date`.

Entregue as queries em `sql/` com o resultado (CSV ou print) e um parágrafo interpretando cada número.

Parte 4 — Qualidade e observabilidade

Implemente ao menos 5 verificações, com severidade e comportamento definidos (o que apenas alerta e o que **para o pipeline**):

- unicidade da chave primária na silver
- integridade referencial (`events.customer_id` → `customers`)
- volumetria fora do esperado em relação aos dias anteriores
- valores de domínio inválidos (`event_type`, `plan`)
- freshness (dado mais recente dentro da janela esperada)

Os resultados devem ser **persistidos** (tabela Iceberg de log de qualidade ou equivalente), não só impressos no terminal. Descreva como você conectaria isso a um alerta real.

Parte 5 — Orquestração

Uma DAG do Airflow representando o fluxo completo, com dependências, `retries`, catchup/backfill e parâmetro de data de execução.

Não é obrigatório que a DAG rode no ambiente. Se você a entregar como código bem estruturado e explicar o desenho no README, conta pontuação integral.

6. Bônus (opcional — não sacrifique o obrigatório por isso)

- Modelagem silver→gold com **dbt** (adapter Spark ou Trino).
 - Testes automatizados (`pytest`) das funções de transformação.
 - GitHub Actions com lint + testes.
 - Manutenção Iceberg: `rewrite_data_files` (compactação), `expire_snapshots`, demonstração de **time travel**.
 - Uso de LLM para enriquecer qualidade de dados — ex.: classificar `properties` de texto livre, ou detectar anomalias — com atenção a custo, latência e determinismo.
-

7. Regras

Uso de IA (Copilot, Claude, ChatGPT) é permitido e incentivado. Trabalhamos com IA no dia a dia. Duas condições: declare no README onde usou, e esteja preparado para explicar **linha por linha** o código entregue. Na conversa de code review vamos pedir alterações ao vivo.

O que NÃO estamos avaliando:

- interface gráfica ou dashboard
- volume de dados (o dataset é pequeno de propósito — queremos ver decisões, não força bruta)

- conhecimento prévio de Iceberg/Trino
- quantidade de linhas de código

8. Como entregar

Repositório Git (público ou privado com acesso para `<usuario>`), contendo:

```
None
├─ README.md           # como rodar, decisões, o que faltou
├─ ARCHITECTURE.md      # ver abaixo
├─ docker-compose.yml
├─ ingestion/
├─ transform/          # jobs PySpark
├─ sql/                # queries Trino + resultados
├─ quality/
├─ dags/
└─ tests/
```

O `README.md` deve permitir que a gente rode tudo com **no máximo 3 comandos**.

O `ARCHITECTURE.md` (1 a 2 páginas) deve responder:

1. Por que você particionou as tabelas desse jeito? Qual seria o impacto de particionar diferente?
2. Como você garantiu idempotência em cada etapa?
3. Como modelou a mudança de plano do cliente ao longo do tempo, e por quê?
4. O que aconteceria com esse pipeline se o volume crescesse 100×? O que quebraria primeiro?
5. O problema de small files no Iceberg: ele existe na sua solução? Como você resolveria?
6. O que você faria diferente com mais uma semana?

9. Critérios de avaliação

Eixo	Peso
Modelagem lakehouse e uso de Iceberg (dedup, MERGE, particionamento, idempotência)	25%

Eixo	Peso
SQL analítico e performance no Trino	20%
Ingestão robusta (retry, incremental, watermark, schema drift)	15%
Qualidade de dados e observabilidade	15%
Engenharia de software (organização, Git, Docker, testes, legibilidade)	15%
Documentação e justificativa das decisões	10%

Após a entrega, faremos uma conversa de **60 minutos de code review**, onde discutiremos suas escolhas. Essa conversa pesa tanto quanto o código.

Boa sorte — e qualquer dúvida sobre o enunciado, pergunte. Saber perguntar faz parte da avaliação.